

# Redis 🦄 (cornudo de mierda)

## Teoría

1. ¿Qué tipo de base de datos es Redis? ¿En qué se diferencia de una base de datos relacional y de otras bases de datos NoSQL como MongoDB?

Redis es una **base de datos NoSQL** de tipo clave/valor, dentro de las diferencias que podemos encontrar con los otros tipos de bases de datos tenemos:

- **Redis** es una base de datos que trabaja en memoria **RAM**, mientras que las bases de datos relacionales o una No SQL orientada a documentos no lo hacen, trabajan en disco.
- La velocidad de las operaciones en Redis es mucho mayor a las de las demás bases de datos, esto debido a que trabaja en **RAM**.

2. ¿Dónde almacena los datos Redis? ¿Qué implicancias tiene esto en términos de velocidad y de persistencia?

**Redis** almacena los datos en **memoria RAM**, esto hace que se pueda acceder a la información de manera rápida y con baja latencia, y la desventaja que tiene es que al depender de la memoria, los datos **PODRÍAN** perderse, y por eso Redis incorpora mecanismos de persistencia.

3. ¿Qué tipos de datos soporta Redis? Listar y describir brevemente cada uno.

### Tipos de Datos:

- **Strings**: Textos o números.
- **Listas**: Son ordenadas (con operaciones para pushar o poppear elementos por lados)
- **Sets**: Conjunto desordenado que garantiza la no duplicidad.
- **Sorted Sets**: Conjunto ordenado que garantiza la no duplicidad.
- **Hash**: Objetos de clave/valor
- **Geospatial**: Permiten representar coordenadas (longitud y latitud).
- Stream
- JSON
- Vector

4. Enunciar las características principales de Redis.

### Características principales:

- Es una base de datos NoSQL clave/valor.

- No posee schema.
- Trabaja en memoria RAM y por lo tanto es muy rápida.
- Persistencia Opcional.

5. Comparar Redis con los RDBMS: ¿en qué casos conviene usar Redis en lugar de una base de datos relacional y en cuáles no?

Es **conveniente usar Redis** cuando se necesita realizar operaciones con una gran velocidad, por ejemplo: Caché, rankings, sesiones, etc.

Una **base de datos relacionales es conveniente** usarla cuando se necesita modelar relaciones complejas entre los datos del dominio.

6. ¿Redis tiene soporte para transacciones? ¿Cómo funcionan? ¿Qué garantías ofrecen y qué limitaciones tienen respecto de las transacciones ACID?

Redis tiene soporte transaccional, pero **no es equivalente al de un RDBMS**.

### Garantías

- Atomicidad sobre una clave/agregado.
- Operaciones ejecutadas en orden.
- No hay interferencia durante la ejecución.

### Limitaciones

- No maneja relaciones entre múltiples entidades como un RDBMS.
- Durabilidad depende de la persistencia configurada.
- En sistemas distribuidos puede priorizar disponibilidad sobre consistencia.

7. ¿Redis tiene persistencia? Describir los mecanismos disponibles (RDB y AOF) e indicar las diferencias entre ellos.

Si, Redis posee 2 mecanismos de persistencia que se pueden utilizar:

- **RDB:**
  - Funciona como una snapshot de los valores actuales de las claves que se realiza cada X cantidad de tiempo.
- **Ventaja**
  - Es un archivo pequeño de recuperación rápida.
- **Desventaja**
  - Si llegara a pasar algo en el intervalo en el que no se tomó todavía la snapshot, perdemos los cambios realizados.

- **AOF:**
  - Registra cada operación de escritura trabajando como si fuera un Log.
  - **Ventaja**
    - Tiene mayor durabilidad ya que se actualiza con cada escritura.
  - **Desventaja**
    - Es un archivo más grande con una recuperación más lenta.

8. ¿Cuáles son los principales casos de uso de Redis en aplicaciones reales?

#### Principales casos de uso de Redis:

- Cachés.
- Chats en Tiempo real.
- Tokens.
- Sesiones de Usuarios.
- Rankings.
- Notificaciones.
- Colas de Mensajes.

---

## Strings

**Comando estándar** para almacenar Strings `SET clave valor`.

1. Agregar una clave package con el valor "Bariloche 3 days".

```
SET package "Bariloche 3 days"
```

2. Agregar una clave user con el valor "Turismo BD2". Obtener el valor de la clave user.

```
SET user "Turismo BD2"  
GET user
```

3. Obtener todas las claves almacenadas actualmente.

```
KEYS *
```

4. Agregar una clave user con el valor "Cronos Turismo". ¿Cuál es el valor actual de la clave user?

```
SET user "Cronos Turismo"  
GET user → "Cronos Turismo"
```

5. Concatenar " S.A." a la clave user. ¿Cuál es el valor actual de la clave user?

```
APPEND user " S.A." // retorna la longitud del string
```

6. Eliminar la clave user. ¿Qué valor retorna si se intenta obtener la clave user luego de eliminarla?

```
DEL user // retorna 1 si existe sino 0  
GET user → nil
```

7. Verificar si existe la clave visits.

```
EXISTS visits // si existe 1 sino 0
```

8. Agregar una clave visits con el valor 0.

```
SET visits 0
```

9. Incrementar en 1 la clave visits. ¿Cuál es el valor actual?

```
INCR visits  
GET visits → 1
```

10. Incrementar en 5 la clave visits. ¿Cuál es el valor actual?

```
INCRBY visits 5
```

11. Decrementar en 1 la clave visits. ¿Cuál es el valor actual?

```
DECR visits
```

12. Incrementar en 2 la clave visits. ¿Cuál es el valor actual?

```
INCRBY visits 2
```

13. Agregar una clave "value package" con el valor 539789.32.

```
SET "value package" 539789.32
```

14. Incrementar en 20000 la clave "value package". ¿Cuál es el valor actual?

```
INCRBYFLOAT "value package" 20000
```

15. ¿Cual es el tipo de datos de "value package", visits y user?

```
TYPE visits → string  
TYPE user // Al haber borrado esta te da None  
TYPE "value package" → string
```

---

## Manejo de Claves

1. Obtener todas las claves que empiecen con "v".

```
KEYS v*
```

2. Obtener todas las claves que contengan la letra "t".

```
KEYS *t*
```

3. Obtener todas las claves que terminan con "age".

```
KEYS *age
```

4. Renombrar la clave "package" por "bariloché package".

```
RENAME package "bariloché package"
```

5. ¿Qué comando se utiliza para renombrar una clave solo si el nombre destino no existe aún?

```
RENAMENX package "bariloché package" // si no se ejecuta retorna 0 sino 1
```

6. Eliminar todas las claves.

```
FLUSHDB
```

---

## Expiración de Claves

1. Agregar una clave agency con el valor "Cronos Tours".

```
SET agency "Cronos Tours"
```

2. ¿Cuál es el tiempo de vida (TTL) de la clave agency?

```
TTL agency // por defecto al crear una clave no tiene TTL (-1)
```

3. Agregar una expiración de 30 segundos a la clave agency.

```
EXPIRE agency 30
```

4. ¿Cuál es el tiempo de vida de la clave agency luego de agregar la expiración?

```
TTL agency // número positivo < 30
```

5. Pasados los 30 segundos: ¿cuál es el TTL de agency? ¿Que retorna si se solicita el valor de agency?

```
TTL agency // retorna -2 cuando se venció
```

6. Agregar una clave agency con el valor "Cronos Tours" que expire en 20 segundos desde su creación.

```
SET agency "Cronos Tours" EX 20
```

---

## Listas

1. Insertar una lista llamada pets con el valor "dog".

```
LPUSH pets dog
```

2. ¿Qué sucede si se ejecuta el comando GET sobre pets? ¿Cómo se obtienen los valores de una lista?

```
GET pets // error pa
```

```
LRANGE pets 0 -1
```

3. Agregar a la lista pets el valor "cat" por la izquierda.

```
LPUSH pets cat
```

4. Agregar a la lista pets el valor "fish" por la derecha.

```
RPUSH pets fish
```

5. ¿Qué tipo de dato es el valor de pets?

```
TYPE pets → list
```

6. Eliminar el valor del extremo izquierdo de la lista.

```
LPOP pets
```

7. Eliminar el valor del extremo derecho de la lista.

```
RPOP pets
```

8. Agregar a una clave "vuelo:ar389" los valores: aep, mdz, brc, nqn y mdq.

```
RPUSH vuelo:ar389 aep mdz brc nqn mdq
```

9. Ordenar los valores de la lista "vuelo:ar389". ¿Qué sucede si se solicitan todos los valores de la lista luego de ordenarla?

```
SORT vuelo:ar389 ALPHA // No modifica la lista
```

10. Insertar el valor "fte" inmediatamente después de "brc".

```
LINSERT vuelo:ar389 AFTER brc fte
```

11. Insertar el valor "ush" inmediatamente antes de "fte".

```
LINSERT vuelo:ar389 BEFORE fte ush
```

12. Modificar el último elemento de la lista por "sla".

```
LSET vuelo:ar389 -1 sla
```

13. Obtener la cantidad de elementos de "vuelo:ar389".

```
LLEN vuelo:ar389
```

14. Obtener el tercer valor de "vuelo:ar389".

```
LINDEX vuelo:ar389 2
```

15. Eliminar el valor "aep" de "vuelo:ar389".

```
LREM vuelo:ar389 1 aep
```

LREM key count element

count > 0: Remove elements equal to element moving from head to tail.

count < 0: Remove elements equal to element moving from tail to head.

count = 0: Remove all elements equal to element.

16. Quedarse únicamente con los valores de las posiciones 3 a 5 de "vuelo:ar389".

```
LTRIM vuelo:ar389 2 4
```

```
LRANGE vuelo:ar389 2 4
```

17. Agregar en "vuelo:ar389" el valor "fte". ¿Cuántas veces aparece ahora en la lista?

```
RPUSH vuelo:ar389 fte
```



# Sets

1. Agregar un conjunto llamado airports con los siguientes valores: eze aep nqn mdz mdq ush fte sla aep nqn brc cpc juju aep tuc eqs

```
SADD airports eze aep nqn mdz mdq ush fte sla aep nqn brc cpc juju aep tuc eqs
```

2. ¿Cuántos valores tiene el conjunto? ¿Por qué puede diferir de la cantidad de valores ingresados?

```
SCARD airports // difiere porque hay repetidos pa
```

3. Listar los valores del conjunto airports.

```
SMEMBERS airports
```

4. Quitar el valor "cpc" del conjunto airports.

```
SREM airports cpc
```

5. Quitar un valor aleatorio del conjunto airports.

```
SPOP airports
```

6. ¿Qué cantidad de valores tiene airports ahora?

```
SCARD airports
```

7. Comprobar si "cpc" es miembro del conjunto airports.

```
SISMEMBER airports cpc
```

8. Mover los valores "sla" y "juju" a un nuevo conjunto denominado noa\_airports.

```
SMOVE airports noa_airports sla juju
```

9. Obtener la unión de los conjuntos airports y noa\_airports. ¿Modifica los conjuntos originales?

```
SUNION airports noa_airports // no modifica los conjuntos originales
```

10. Realizar la unión de airports y noa\_airports y almacenar el resultado en un nuevo conjunto llamado total\_airports.

```
SUNIONSTORE total_airports airports noa_airports
```

11. Realizar la intersección entre total\_airports y noa\_airports.

```
SINTER total_airports noa_airports
```

12. Realizar la diferencia entre total\_airports y noa\_airports.

```
SDIFF total_airports noa_airports
```

---

## ZSets (Sorted Sets)

1. Agregar a un conjunto ordenado llamado passengers los siguientes datos (score nombre):

```
2.5 federico 4 alejandra 3 julian 1 ivan 2 tomas 2 luciana 2.4 natalia
```

```
ZADD passengers 2.5 federico 4 alejandra 3 julian 1 ivan 2 tomas 2 luciana  
2.4 natalia
```

2. Obtener los valores del conjunto passengers.

```
ZRANGE passengers 0 -1
```

3. Actualizar el score de luciana a 2.7.

```
ZADD passengers 2.7 luciana // sobreescribe el valor
```

4. Agregar al conjunto passengers a silvia con score 5.1.

```
ZADD passengers 5.1 silvia
```

5. Incrementar en 2 el score de alejandra.

```
ZINCRBY passengers 2 alejandra
```

6. Obtener los valores del conjunto passengers con sus scores.

```
ZRANGE passengers 0 -1 WITHSCORES
```

7. Obtener los valores del conjunto passengers con sus scores en orden inverso.

```
ZREVRANGE passengers 0 -1 WITHSCORES
```

8. Obtener la cantidad de elementos del conjunto passengers.

```
ZCARD passengers
```

9. Obtener la cantidad de elementos que tienen scores entre 2 y 3.

```
ZCOUNT passengers 2 3
```

10. Obtener el ranking de julian en el conjunto passengers.

```
ZRANK passengers julian
```

11. Obtener el score de tomas en el conjunto passengers.

```
ZSCORE passengers tomas
```

12. Extraer el elemento con menor score del conjunto passengers.

```
ZPOPMIN passengers
```

13. Extraer el elemento con mayor score del conjunto passengers.

```
ZPOPMAX passengers
```

14. Eliminar del conjunto passengers al valor silvia.

```
ZREM passengers silvia
```

---

## Hashes

1. Agregar a un hash llamado user:cronos los siguientes campos: "razon social" "cronos s.a" domicilio "47 236 La Plata" "telefono" 2215556677

```
HSET user:cronos "razon social" "cronos s.a" domicilio "47 236 La Plata" "telefono" 2215556677
```

2. Agregar el campo mail con el valor [info@cronos.com.ar](mailto:info@cronos.com.ar) al hash user:cronos.

```
HSET user:cronos mail info@cronos.com.ar
```

3. Obtener todos los campos y valores de user:cronos.

```
HGETALL user:cronos
```

4. Obtener únicamente el valor del campo mail de user:cronos.

```
HGET user:cronos mail
```

5. Eliminar el campo teléfono de user:cronos.

```
HDEL user:cronos telefono
```

6. Obtener la cantidad de campos de user:cronos.

```
HLEN user:cronos
```

7. Obtener las claves (nombres de campos) de user:cronos.

```
HKEYS user:cronos
```

8. Determinar si existe el campo cuil en user:cronos.

```
HEXISTS user:cronos cuil
```

9. Obtener todos los valores (sin los nombres de campos) de user:cronos.

```
HVALS user:cronos
```

10. Obtener la longitud del valor del campo mail de user:cronos.

```
HSTRLEN user:cronos mail
```

---

## Geospatial

1. Agregar en un conjunto denominado cities las siguientes localidades con sus coordenadas (longitud, latitud): SON UNA BANDA

```
GEOADD cities -58.37 -34.61 "Buenos Aires" // ejemplo con uno
```

2. Obtener todos los miembros del conjunto cities.

```
ZRANGE cities 0 -1
```

3. Obtener las coordenadas almacenadas de Santa Fe.

```
GEOPOS cities "Santa Fe"
```

4. Obtener la distancia en kilómetros entre Buenos Aires y Córdoba.

```
GEODIST cities "BA" "Cordoba" km // m, km, mi, ft
```

5. Obtener las ciudades que se encuentran en un radio de 100 km de la coordenada (-27.37, -55.9), incluyendo la distancia de cada una.

```
GEOSEARCH cities FROMLONLAT -55.9 -27.37 BYRADIUS 100 km WITHDIST
```

6. Obtener las ciudades que se encuentran a menos de 700 km de Córdoba.

```
GEOSEARCH cities FROMMEMBER "Cordoba" BYRADIUS 700 km
```

## Ejercicios de Claudio 🧐

### Strings

1. Almacenar el precio del paquete "paquete:ushuaia" con el valor 85000.50 . Incrementarlo en 12500.75 . ¿Cuál es el valor final?

```
SET paquete:ushuaia 85000.50
INCRBYFLOAT paquete:ushuaia 12500.75
GET paquete:ushuaia
```

2. Guardar la clave "reservas:activas" con valor 0 . Simular 3 reservas nuevas incrementando de a 1. Luego cancelar 2 reservas. ¿Cuál es el valor final?

```
SET reservas:activas 0
INCR reservas:activas
INCR reservas:activas
INCR reservas:activas
DECR reservas:activas
DECR reservas:activas
GET reservas:activas
```

2. Agregar la clave "agencia:nombre" con valor "Patagonia" . Concatenarle " Viajes & Turismo S.A." . ¿Cuántos caracteres tiene el valor final? ¿Con qué comando lo verificás?

```
SET agencia:nombre Patagonia
APPEND agencia:nombre " Viajes & Turismo S.A." // este te da la cant de caracteres
```

### Listas

4. Crear la lista "escala:vuelo:ar201" con los aeropuertos: eze mdz sla juj . Insertar "tuc" entre "sla" y "juj" . Luego quedarse únicamente con los últimos 3 elementos.

```
RPUSH escala:vuelo:ar201 eze mdz sla juj
INSERT escala:vuelo:ar201 AFTER sla tuc
LTRIM escala:vuelo:ar201 -3 -1
```

5. Tenés la lista "cola:check-in" con pasajeros: "ana" "luis" "marta" "pedro" "carlos". Simular que se atienden los primeros 2 pasajeros (eliminándolos). Agregar un pasajero "julia" al final. ¿Cuántos pasajeros quedan en la cola?

```
LPOP cola:check-in
LPOP cola:check-in
RPUSH cola:check-in julia
LLEN cola:check-in
```

## Sets

6. Crear el conjunto "destinos:verano" con: mdq mdo brc bue ush fte. Crear el conjunto "destinos:invierno" con: brc ush sla juj mdz bue. Obtener los destinos que se ofrecen en **ambas** temporadas. Obtener los destinos que son **exclusivos** del verano.

```
SADD destinos:verano mdq mdo brc bue ush fte
SADD destinos:invierno brc ush sla juj mdz bue
SINTER destinos:verano destinos:invierno
SDIFF destinos:verano destinos:invierno
```

7. Tenés los conjuntos "vuelo:ar100:pasajeros" y "vuelo:ar200:pasajeros" con distintos nombres. Un pasajero "marcos" cambió su vuelo del ar100 al ar200. ¿Con qué comando único hacés ese cambio?

```
SMOVE vuelo:ar100:pasajeros vuelo:ar200:pasajeros marcos
```

## ZSets

8. Crear el sorted set "ranking:agencias" con los siguientes scores (ventas en miles): agencia\_norte 120 agencia\_sur 95 agencia\_centro 140 agencia\_este 80 agencia\_oeste 110. Obtener el ranking **de mayor a menor** con sus scores. ¿Cuál es el puesto de agencia\_norte?

```
ZADD ranking:agencias 120 agencia_norte 95 agencia_sur 140 agencia_centro 80
agencia_este 110 agencia_oeste
```

```
ZREVRANGE ranking:agencias 0 -1 WITHSCORES
ZREVRANK ranking:agencias agencia_norte
```

9. Sobre el conjunto anterior: obtener solo las agencias con ventas **entre 100 y 140**. Luego incrementar en 30 las ventas de `agencia_sur`. ¿Cambia su posición en el ranking?

```
ZRANGEBYSCORE ranking:agencias 100 140
ZINCRBY ranking:agencias 30 agencia_sur
ZREVRANK ranking:agencias agencia_sur
```

## Hashes

10. Crear el hash `"vuelo:ar389"` con los campos: `origen eze destino mdz duracion 120 precio 45000 asientosDisponibles 15`. Reducir en 1 los asientos disponibles (como si alguien comprara un pasaje). ¿Con qué comando lo hacés sin tener que hacer GET y luego SET?

```
HSET vuelo:ar389 origen eze destino mdz duracion 120 precio 45000
asientosDisponibles 15
HINCRBY vuelo:ar389 asientosDisponibles -1
```

11. Tenés el hash `"hotel:patagonia"` con campos: `nombre, categoria, precio, ciudad, mail`. Obtener únicamente los **nombres de los campos** sin sus valores. Luego verificar si existe el campo `"telefono"`.

```
HKEYS hotel:patagonia
HEXISTS hotel:patagonia telefono
```

## Geospatial

12. Registrar en el conjunto `"hoteles"` los siguientes hoteles con sus coordenadas:

- Hotel Bariloche: `-71.30, -41.13`
  - Hotel Mendoza: `-68.82, -32.88`
  - Hotel Ushuaia: `-68.30, -54.80`
- Obtener todos los hoteles que estén a menos de **1500 km** de Hotel Mendoza, mostrando la distancia de cada uno.

```
GEOADD hoteles -71.30 -41.13 "Hotel Bariloche" -68.82 -32.88 "Hotel Mendoza"
-68.30 -54.80 "Hotel Ushuaia"
```



```
GEOSEARCH hoteles FROMMEMBER "Hotel Mendoza" BYRADIUS 1500 km WITHDIST
```

## Expiración (combinado)

13. Crear la clave "oferta:black-friday" con valor "50% off paquetes Caribe" que expire automáticamente en **60 segundos** desde su creación. A los 20 segundos, ¿cómo verificás cuánto tiempo de vida le queda? ¿Qué devuelve GET una vez expirada?

```
SET oferta:black-friday "50% off paquetes Caribe" EX 60  
TTL oferta:black-friday  
GET oferta:black-friday → nil
```